Project and Demo Inventory

Your Name

October 2, 2024

Projects and Demos

Embedded Common C and C++

- **Static heap allocators**: Dynamic allocation semantics for advanced data structures such as open address hash tables, RB trees, and graphs.
- Compatible with `std::c++`.
- C++14 conforming `stevemac::vector` with static allocator.

STM32CubeIDE

- Interrupt bare metal "multi-threading" with `std::atomics`.
- System Graph.
- STM32CubeMX pinout configuration.
- Semihosting.

Keil uVision with STM32CubeMX

- STM32CubeMX pinout config integrated.
- Interrupt bare metal "multi-threading" with C11 atomics.

VSCode

- Rust embedded bare metal state captures.
- Semihosting.
- Cortex Quick Start for STM32F401RE.

Visual Studio

- Raspberry Pi remote cross compile and debugging.

MLibs

- Embedded automation suite.
- Semihosting.

Raspberry Pi Assembly

- Native assembly development (sorts and other routines).

Algorithms C++ General

- Implemented data structures and algorithms: Automata, Graph, Linked List, Stack, Queue, Heap Sort, Merge Sort, Quick Sort, Open address hash table with cluster diagnostics, Red Black Tree.
- C++14 conforming `stevemac::vector` with static allocator.
- Multithreaded, SIMD matrix multiplication.
- Rust code generator simple "wizard".
- FreeBSD/Linux Tree and List macros.

Algorithms Rust

- Implemented algorithms in Rust: Kadane, Linked List, Binary Tree, Quick Sort, Valid Parentheses, Binary Search, Insertion Sort, String Permute, Tic-tac-toe, etc.

Embedded Systems Project: ADC, DSP, and FFT Demonstration

Project Overview

The project involves an ADC (Analog-to-Digital Conversion), DSP (Digital Signal Processing), and FFT (Fast Fourier Transform) demo using custom data structures such as a priority queue and graph. This demonstration showcases advanced data structure usage in an embedded environment.

Steps and Basis for Edge Connections

1. **Priority Queue Construction**: The FFT output is stored in a priority queue using `priority_queue_from_array` and `priority_queue_build_max_heap`. The heap is then printed with `priority_queue_print_heap`, organizing the FFT magnitudes based on their values (peaks and valleys).
2. **Graph Creation**: A graph is created with `create_graph(&graph, FFT_LENGTH)`, initializing the graph structure with a size based on the FFT length. Each vertex represents a magnitude from the FFT results.
3. **Vertices Addition**: For each value in the heap (`pq.heap`), a vertex is added to the graph. Each vertex corresponds to an FFT output magnitude.
4. **Edge Creation Based on Peaks and Valleys**: The core logic for adding edges checks for peaks and valleys in the magnitude values of the FFT results. Peaks are connected to neighboring valleys and vice versa.
5. **Graph Visualization**: The graph is printed using `print_graph(&graph)`, showing the vertices (peaks and valleys) and their connections (edges).

Edge Connection Summary

- Peaks are connected to adjacent valleys and vice versa.
- The edges are added based on relative magnitude comparisons of adjacent FFT results.

FreeBSD Kernel Deep Dive

- 45 hours of FreeBSD kernel study with McKusick's deep dive course.

Rust Programming

- Completed the Rust book cover-to-cover, covering ownership, borrowing, smart pointers, concurrency, and more.

Embedded Systems and Development Tools

Languages

- C, C++, ARM Assembly.
- Rust (embedded development).

Embedded Platforms

- ARM Cortex M series: STM32, Tiva C microcontrollers.
- Raspberry Pi.

Development Tools

- IAR Embedded Workbench, Keil uVision.
- STM32CubeMX (code generator).
- STM32CubeIDE, uVision Runtime Manager.
- Code Composer Studio, PinMux, Resource Explorer, Driverlib.
- Visual Studio, VisualGDB, VSCode, Vim, NeoVim.
- Rust embedded crates, ST-Link, OpenOCD.
- CLI environment (gdb + stlink or openocd).
- STM32 HAL, CMSIS Programming, Legacy StdPeripheral.
- MISRA Warnings enabled.

Embedded Programming Skills

- Advanced algorithms: Fast Fourier Transform, Automata, Finite State Machines.
- Electronic sampling: analog-to-digital, digital-to-analog, temperature, and voltage sampling.
- Data integrity techniques: wear leveling, CRC.
- Custom allocators using fixed array-based heap managers (e.g., Valvano).
- Standard C++ Library containers (e.g., `std::vector`) with fixed array-based custom allocators.
- Data structures optimized for embedded environments (stacks, queues, circular buffers, binary trees, priority queues, heaps, graphs, open-address hash tables).
- Algorithms derived from *Introduction to Algorithms* by Cormen, Leiserson, Rivest, and Stein, Sedgewick's *Algorithms*, and *Foundations of Computer Science* by Aho and Ullman.
- Automated embedded testing using CLI-based build, flash, and test sequences.
- Integration with STM32CubeMX for STM32 and PinMux/Resource Explorer for Tiva C series.
- CMSIS DSP Libraries for signal processing and testing.

Embedded Systems Skills

- General Operating Systems knowledge, Computer Organization, Binary Structure, and Embedded Startup Code Sequences.
- Timers and Clocks, generated programmatically or using tools like STM32CubeMX.
- Interrupt Handling, Fault Monitoring, and Recovery.
- Peripheral Communication: GPIO, UART, SSI, I2C, Timer, PWM, ADC, Ethernet, CAN.
- DMA streaming, Memory Management, Flash Memory, EEPROM, Power Optimization.
- Atomic Operations, Concurrency, and Parallelism (C++ Coroutines, Rust's async/await).
- Debugging Tools: Keil uVision, STM32CubeIDE, Code Composer Studio, VisualGDB, CLI (gdb + st-link or openocd).
- Semihosting, UART Printing, ITM Tracing.